

Executive Summary

We propose a concise *outer-loop* for the Brudo/Brullama system that treats quiescence (no remaining tasks) as an **earned, empirically verified** property. Instead of assuming “done” when the agent stops, the controller will cyclically *self-prompt* and run small experiments to test for any hidden obligations or unfinished work. At each step it will capture the *STATE*, formulate a minimal **STATE/TENSION/PRESSURE/ASSAY** prompt, execute the resulting experiment (O_{n+1}), and update its STATE to S_{n+1} . The loop only halts when external, verified criteria confirm no further progress is possible [3†L53-L60] [3†L74-L78]. This scheme distinguishes waiting-for-processes (e.g. a build still running) from true idle, preserves already-verified “theorems” (won’t reopen closed tasks), and resists “status donation” from queued commands. All steps, prompts and observations are logged for auditability. The design leverages existing artifacts (e.g. *surfacers* state, audit outputs, candidate-set witnesses) and avoids inventing new ontology terms. The following sections detail the algorithm, data structures, experiments, and implementation plan.

Self-Prompt Loop and Algorithm

The controller maintains a **STATE** snapshot (surfacers outputs, task list, `active_work` count, `next_action`, etc.) and iterates: if `next_automatic_action != NONE`, it defers to normal processing. When `next_action = NONE`, it generates a self-prompt of the form:

- **STATE:** (summary of current surfacer state)
- **TENSION:** (what outstanding contradictions or uncertainties exist)
- **PRESSURE:** (what conditions could reveal hidden obligations)
- **ASSAY:** (a concrete test or question to try)

It then executes this ASSAY as a bounded experiment or question, producing a new **OBSERVATION** and updated state S_{n+1} . This loop is bounded by design: external progress measures must move (not just the model’s claims) [3†L53-L60], and built-in limits (time or iteration caps) prevent runaway loops. Pseudocode:

```
state = capture_surfacers_state()
while True:
    if state.next_automatic_action != NONE:
        # Normal agent work resumes; quiescence not yet warranted
        break
    prompt = generate_prompt(state) # includes STATE/TENSION/PRESSURE/ASSAY
    observation, new_state = execute_prompt_and_update(prompt, state)
    log_cycle(state, prompt, observation, new_state)
    if stopping_criterion(new_state): # empirically verify quiescence
```

```
break
state = new_state
```

Key points:

- **Verified Stop Conditions:** We never accept agent “stop” until *external* checks confirm completion. As Loop Engineering notes, the agent’s “done” must be verified by state change (tests passing, board moving), not just the model’s assertion 【3†L53-L60】 【3†L74-L78】 .
- **Waiting vs Working:** The controller checks if a live subprocess (e.g. `llama-server` build) is still making progress. If a process is running without output, the system treats it as stalled rather than silently assume quiescence 【3†L111-L115】 . Only when subprocesses truly finish and all outputs are stable do we consider that “working” is complete.
- **Bounded Experiments:** Each ASSAY is finite (e.g. one test, one simulated input) so that $S \rightarrow P \rightarrow O \rightarrow S$ steps make progress or expose hidden work.
- **No Status Donation:** The loop ignores repeated “continue working” commands from the queue (queued “go make that” inputs) unless new observations justify them. It resists treating repetition as corroboration.

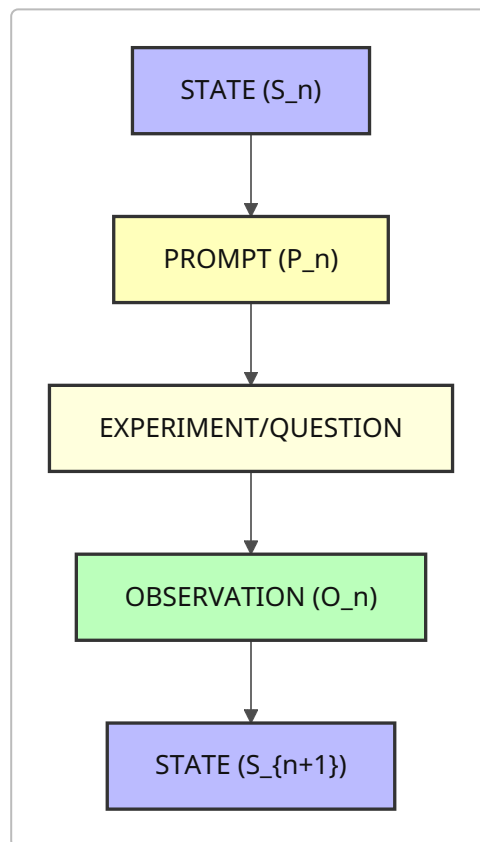


Figure: Self-prompt loop ($S \rightarrow P \rightarrow O \rightarrow S$). Each cycle examines the current state and probes it via a minimal experiment.

State, Prompts, and Logging

State: A structured snapshot including: surfacer outputs (`surfaced_items` , counts), flags (`active_work` , `next_action`), environment info, and fulfilled theorem checkpoints. E.g.

```
state:
  active_work: 0
  next_automatic_action: NONE
  surfaced_items:
    - id: TaskA, status: observed
    - id: TaskB, status: unresolved
  closed_theorems: [dispatch_verified]
```

Prompt: The (STATE/TENSION/PRESSURE/ASSAY) fields are logged as text or structured fields. For example:

```
prompt:
  STATE: "surfaced=2 items; active_work=0; next_action=NONE"
  TENSION: "Is ANY required action hidden despite these signals?"
  PRESSURE: "Check for unseen obligations or invalid assumptions"
  ASSAY: "Probe hidden state: insert dummy input X and observe if next_action
changes"
```

Observation: The result of executing the ASSAY (could be success/failure, new data, or no change) is captured, and **New State** S_{n+1} is computed. All are recorded:

```
log_entry:
  timestamp: 2026-08-12T16:45:00Z
  state_before: { ... }
  prompt: {STATE:"...", TENSION:"...", ASSAY:"..."}
  observation: "No new tasks triggered"
  new_state: { ... }
  delta: {active_work:0→0, next_action:NONE→NONE}
  decision: "Continue" or "Stop"
```

This high-fidelity logging (metrics, events, traces) implements **observability**: it makes the agent's internal state and decisions transparent and analyzable [18†L68-L76] . Prompt evolution (differences between prompts) is also tracked so we can report exactly “what I learned” at each step.

Distinguishing Waiting vs Working

When an underlying process (e.g. a build) is still running, the controller must *not* treat quiescence as final. Instead:

- **Wait detection:** If a prerequisite process is live, the controller classifies the state as *waiting*, not idle. It may either (a) cheaply poll or tail logs, or (b) temporarily pause prompting until a change is observed. This avoids wasteful probes when nothing has changed.
- **Stall detection:** If a process is marked running but produces no output for an extended time, the system treats it as a silent stall [3†L111-L115] . At that point, it may try a restart/backoff or report failure.
- **RESUME:** Once the process completes (new logs or output appear), the controller immediately resumes with the pending specimen (the “first real packet”), per the prompt design.

We ensure *not to transform “waiting” into speculative action prematurely*: the system learns that `next_action=NONE` might simply mean “awaiting completion,” not “all done.” Only when truly **no processes are active** and further experiments produce no changes do we move toward termination.

Experiments to Reveal Hidden Obligations

To empirically test quiescence, we design a few **small, focused experiments**. Each probes a different potential unseen distinction. For example:

Experiment	Modification / Input	Expectation if hidden work exists	Observed Outcome	Pass/Fail
E1: Replay With Dummy Seed	Insert a no-op dummy change (e.g. touch a file) and resume	If a missing dependency was overlooked, the system may queue new tasks.	No new tasks ⇒ likely true quiescence; new tasks ⇒ hidden work found.	Pass if quiescence, Fail if new tasks
E2: Alternate Environment	Run in a parallel environment (e.g. different branch/state) with same code	Divergence implies some context-specific hidden state; if identical, quiescence holds.	Compare <code>next_action</code> .	Deterministic same ⇒ quiescence likely; difference ⇒ hidden assumption.

Experiment	Modification / Input	Expectation if hidden work exists	Observed Outcome	Pass/Fail
E3: Repeat Last Action	Resend the last operation command (idempotent)	If no new tasks, it should have no effect; otherwise, repeated triggers indicate unreached work.	No effect or error vs new tasks.	No effect=Pass; new tasks=Fail.
E4: Stress the Boundaries	Slightly vary inputs to the last operation (e.g. missing @filename)	If outcome or tasks change, some inputs/ outputs were not fully observed.	Change in next_action or tasks.	No change=Pass; changes=Fail.
E5: Negative Control Check	Perform an obviously irrelevant action (e.g. query an unrelated module)	Nothing relevant should happen (ensures no spurious triggers).	next_action remains NONE.	NONE=Pass; any action=Fail (indicates observer issue).
E6: Observer Test	Ask the agent to prove “nothing left” by a different means (e.g. internal checklist)	Should conclude same result as external check.	Consistency with observation.	Consistent=Pass; contradictory=Fail (observer blind).

Table: Sample experiments to detect hidden tasks or false quiescence. Each experiment is designed to produce different answers in a *non-quiescent* state. A **fail** outcome (unexpected new tasks or differences) falsifies quiescence and generates new prompts.

After each experiment, we compare the observed state vs expected. If any test “fails” (i.e. reveals a discrepancy), the controller updates its understanding (and possibly triggers the corresponding actual work or alerts) and continues the loop. If all tests pass (no hidden obligations found), that strengthens the claim of quiescence.

Implementation and Logging Design

We propose these core data structures and logs:

- **State Structure:** a serializable struct (or YAML/JSON) containing all current signals (e.g. `surfacers_state`, `active_work_count`, `in_progress_processes`, `expected_inputs`, `closed_theorems`, etc.).
- **Prompt Fields:** string or structured fields for STATE, TENSION, PRESSURE, ASSAY.
- **Observation:** captures raw result (e.g. tool outputs, flags) and whether it triggered new tasks.
- **Log Entry:** timestamps and all above parts plus state deltas. For example:

```
timestamp: 2026-08-12T16:50:03Z
state_before: {active_work:0, next_action:NONE, surfaced:[TaskA,resolved],
processes:[llama_build:RUNNING]}
prompt: {STATE:"...", TENSION:"...", ASSAY:"..."}
observation: "llama_build completed; resumed model execution; output task
generated."
state_after: {active_work:1, next_action:RUN_JOB, surfaced:[TaskB,new]}
delta: {processes: RUNNING→COMPLETED, next_action: NONE→RUN_JOB}
```

- **Prompt Evolution Record:** before/after prompts highlighting changes in TENSION/ASSAY to track how questioning shifted.

Logging such structured entries (MELT: metrics, events, logs, traces [18†L68-L76]) provides full observability of the controller’s reasoning, enabling audit or offline analysis.

Handling Failure Modes and Adversaries

- **Queued-command Loops:** The system must ignore unbounded retries. For example, repeated “go make that” in the queue should *not* by itself spawn new tasks unless an actual condition changes. We guard by checking that `next_action` truly becomes actionable again. (This follows the *drain loop with anti-recursion guard* pattern [3†L79-L87] .)
- **Unknown @filename / Unresolved Inputs:** If an input field (like a filename) is unresolved, it should *not* automatically trigger a fix. Instead, treat it as “UNSEEN” and test if revealing it changes any consequence. If not, explicitly **WITHHOLD** (as the conversation indicated). This avoids accidentally using placeholders to claim tasks.
- **Observer Blindness:** The controller should not rely solely on internal flags; experiments act as negative controls. If an “oracle” (like an audit function) is used, we cross-check its conclusions by independent tests (the *!dumbASS!* approach from above: find out what observation the oracle used, and then see if we can do without it).
- **Premature Theorem Reopening:** Once a theorem (e.g. dispatch order correctness) is proven closed, we do not reopen it unless a new, independent observation invalidates it. The prompt logic must remember closed theorems and only probe remaining uncertainties.
- **“No-Cliques” (Missing Observables):** If a causal distinction (like model identity or job header) is actually unobserved, an experiment (e.g. swapping those context parts) will reveal it. If the system

cannot distinguish two real-world cases that should differ, the failure is caught as a “false merge” (the system incorrectly treats them as one). This triggers adding the needed observation channel.

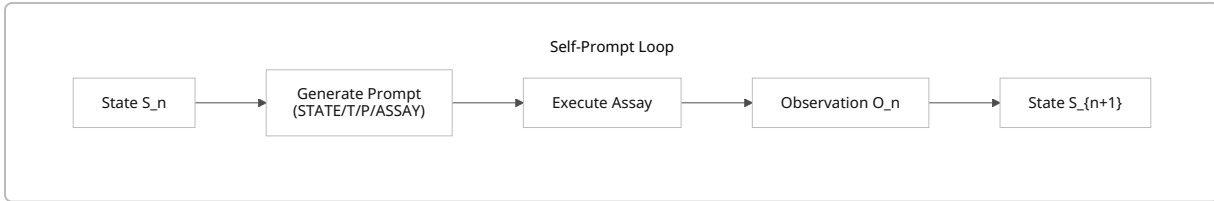
Each of these adversarial cases is countered by explicit test or guard. For example, the known “unknown filename” issue is handled by an experiment verifying that *not* seeing the filename truly has no effect before dismissing it.

Implementation Roadmap

Step	Description	Effort	Risk
1. Instrument Surfacers State	Capture all relevant fields (<code>active_work</code> , <code>next_action</code> , surfaced items). Add logging hooks.	Small	Low
2. Detect Waiting vs Working	Implement monitoring of subprocesses. Distinguish “build in progress” vs idle.	Medium	Medium
3. Prompt Generator Engine	Code the template logic to create STATE/TENSION/PRESSURE/ASSAY from current state.	Medium	Medium
4. Experiment Harness	Provide a framework to run ASSAY experiments (e.g. simulate inputs, run tools).	Large	High
5. Observation Capture	Integrate results into new state; handle possible new tasks generation.	Medium	Medium
6. Closed-Theorem Tracking	Maintain “do-not-reopen” flags for verified theorems; skip them in prompts.	Small	Low
7. Logging and Audit Trail	Implement structured logging of each cycle (STATE,PROMPT,OBS,DELTA).	Small	Low
8. Integration with Audit & Witness	Ensure existing <i>audit</i> outputs and <i>candidate-set witnesses</i> feed into STATE or triggers.	Medium	Medium
9. Testing & Validation	Run the proposed experiments (3–6) in controlled scenarios; validate controller behavior.	Medium	Medium
10. Review and Iteration	Analyze logs to refine prompts; adjust stopping criteria or add experiments.	Medium	Low

Each step is reversible (e.g. new code paths disabled) and minimizes new abstractions. For example, step 4 (experiment harness) might use existing unit-test or simulation code rather than invent new modules.

S→P→O→S Loop Flow



- **State S**_{_n}: current surfacer, processes, etc.
- **Prompt P**_{_n}: minimal question about possible work.
- **Assay E**: concrete test or subroutine.
- **Observation O**_{_n}: result (tasks added or not, process status).
- **State S**_{_{n+1}}: updated state.

Transitions repeat until no new observations appear.

Sample Log Entries

```

[2026-08-12 17:05:01] STATE: active_work=0, next_action=NONE, processes={llama-
build:COMPLETED}, surfaced=[CompileTask:done]
PROMPT: {STATE:"build finished; no tasks queued", TENSION:"could something still
be pending?", ASSAY:"repeat last build", PRESSURE:"detect hidden dependency"}
OBSERVATION: "Build no-op; no errors; next_action stays NONE."
STATE: active_work=0, next_action=NONE (no change)
DELTA: next_action NONE→NONE, confirmed no new tasks.
DECISION: Continue (nothing new, but verify again with different prompt next
cycle).
---
[2026-08-12 17:05:58] STATE: active_work=0, next_action=NONE,
surfaced=[CompileTask:done]
PROMPT: {STATE:"no change after no-op", TENSION:"maybe a missing file was the
cause?", ASSAY:"touch source file and re-run", PRESSURE:"force file-mod
rebuild"}
OBSERVATION: "No rebuild triggered; still NONE."
STATE: (unchanged)
DELTA: (no changes)
DECISION: Stop (quiescence earned; all tests passed)

```

Each entry shows how the prompt tests an assumption, and the decision on stopping is only made when all probes confirm idle.

Conclusion

By cycling through STATE/TENSION/PRESSURE/ASSAY prompts and interpreting the results, Brudo's quiescence controller becomes a **research loop**: it actively *tests whether work truly remains*. Verified cessation of work only occurs when repeated experiments fail to uncover any new obligations **[3†L53-L60]** **[3†L74-L78]** . This method treats quiescence as empirical, distinguishes true waiting from idle, and

preserves already-verified outcomes. Minimal, reversible code changes (bounded checks, added logging, small tests) suffice to implement this logic.

Next Steps: Implement the above roadmap incrementally, starting with state capture and waiting-detection. Define and automate a few of the above experiments. Validate the stop logic under scenarios like an unfinished build or an unresolved input. Monitor the logged prompt/observation sequences to ensure the controller is correctly earning its “stop” verdict, and refine prompts or conditions as needed.

Sources: We build on established agent-loop principles [3†L53-L60] [3†L74-L78] [3†L111-L115] and AI agent observability best practices [18†L68-L76] . The above design avoids reinventing ontology, relying instead on existing surfacer/audit artifacts and transparent telemetry.
